HOSTEL MANAGEMENT SYSTEM

DBMS MINI PROJECT REPORT

DATABASE MANAGEMENT SYSTEMS (DBMS)

Submitted by:

Nilansha Srivastava (202501100100169)

Riya Dwived(202501100100215)

Under the guidance of:

Ms. Prerna Tyagi

The Hostel Management System is a relational database mini project that organises information about hostels, wardens, rooms, students and payments. It uses Oracle SQL to create related tables, define primary and foreign keys, apply integrity constraints, insert sample records and retrieve information through SQL queries. The project demonstrates one-to-many relationships, a composite primary key, joins, aggregate functions and subqueries.

Table of Contents

- Introduction
- Problem Statement
- Objectives and Scope
- Tools and Technologies
- ER Model and Relationships
- Relational Schema
- Database Implementation
- Sample Data
- SQL Queries
- Testing and Validation
- Limitations
- Future Scope
- Conclusion
- References

1. Introduction

Hostel administration requires maintaining information about accommodation facilities, rooms, wardens, students and fee payments. Manual record keeping can make searching, updating and verifying information difficult. This project stores these records in related tables so that information can be retrieved consistently using SQL.

2. Problem Statement

Design and implement a relational database that stores hostel-related information and maintains valid relationships between hostels, wardens, rooms, students and payments. The database should reduce unnecessary duplication, preserve data integrity and support common administrative queries.

3. Objectives and Scope

• Store hostel details and assigned wardens.

• Maintain room floor, capacity and AC status.

• Maintain student details and room allocation.

• Record multiple payment transactions for a student.

• Enforce data integrity using primary keys, foreign keys and validation constraints.

• Retrieve information using joins, grouping, aggregate functions and subqueries.

Scope: The current project focuses on the database layer and SQL operations. A complete web/mobile interface and real payment gateway are outside the current implementation.

4. Tools and Technologies

• Oracle SQL / Oracle FreeSQL

• GitHub for source-code version control

• DBMS concepts: relational model, keys, constraints, joins and subqueries

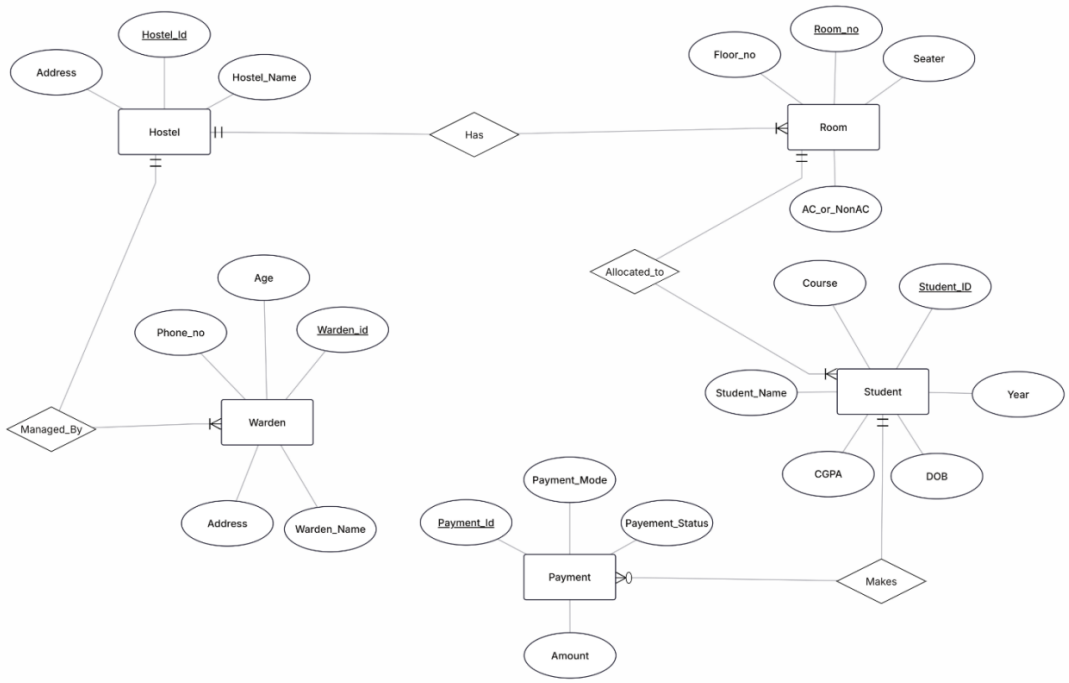
5. ER Model and Relationships

• HOSTEL–WARDEN (1:N): One hostel can have multiple wardens; each warden is assigned to one hostel.

• HOSTEL–ROOM (1:N): One hostel contains multiple rooms; each room belongs to one hostel.

• ROOM–STUDENT (1:N): One room can accommodate multiple students; each student is allocated to one room.

• STUDENT–PAYMENT (1:N): One student can have multiple payment records; each payment belongs to one student.



ER diagram supplied for the project..

6. Relational Schema

Table	Primary Key	Other attributes / Foreign Keys
HOSTEL	Hostel_Id	Hostel_Name, Address
WARDEN	Warden_Id	Warden_Name, Age, Phone_No, Address, Hostel_Id (FK)
ROOM	(Hostel_Id, Room_No)	Floor_No, Seater, AC_or_NonAC; Hostel_Id (FK)
STUDENT	Student_Id	Student_Name, Course, Year, CGPA, DOB, Hostel_Id + Room_No (composite FK)
PAYMENT	Payment_Id	Student_Id (FK), Payment_Mode, Payment_Status, Amount

ROOM uses a composite primary key (Hostel_Id, Room_No), because room numbers can repeat across hostels. STUDENT references both columns to identify the allocated room.

7. Database Implementation

Create parent tables before dependent tables: HOSTEL, then WARDEN and ROOM, followed by STUDENT and PAYMENT. The following Oracle SQL defines the tables and constraints.

```
CREATE TABLE HOSTEL (
  Hostel_Id NUMBER(5) PRIMARY KEY,
  Hostel_Name VARCHAR2(50) NOT NULL,
  Address VARCHAR2(100)
);

CREATE TABLE WARDEN (
  Warden_Id NUMBER(5) PRIMARY KEY,
  Warden_Name VARCHAR2(50) NOT NULL,
  Age NUMBER(3) CHECK (Age >= 18),
  Phone_No VARCHAR2(15) UNIQUE,
```

```
  Address VARCHAR2(100),
  Hostel_Id NUMBER(5) NOT NULL,
  CONSTRAINT fk_warden_hostel FOREIGN KEY (Hostel_Id)
    REFERENCES HOSTEL(Hostel_Id)
);

CREATE TABLE ROOM (
  Hostel_Id NUMBER(5),
  Room_No NUMBER(5),
  Floor_No NUMBER(2),
  Seater NUMBER(2) CHECK (Seater > 0),
  AC_or_NonAC VARCHAR2(10)
    CHECK (AC_or_NonAC IN ('AC', 'Non-AC')),
  CONSTRAINT pk_room PRIMARY KEY (Hostel_Id, Room_No),
  CONSTRAINT fk_room_hostel FOREIGN KEY (Hostel_Id)
    REFERENCES HOSTEL(Hostel_Id)
);

CREATE TABLE STUDENT (
  Student_Id NUMBER(5) PRIMARY KEY,
  Student_Name VARCHAR2(50) NOT NULL,
  Course VARCHAR2(30),
  Year NUMBER(1) CHECK (Year BETWEEN 1 AND 5),
  CGPA NUMBER(3,2) CHECK (CGPA BETWEEN 0 AND 10),
  DOB DATE,
  Hostel_Id NUMBER(5) NOT NULL,
  Room_No NUMBER(5) NOT NULL,
  CONSTRAINT fk_student_room FOREIGN KEY (Hostel_Id, Room_No)
    REFERENCES ROOM(Hostel_Id, Room_No)
);

CREATE TABLE PAYMENT (
  Payment_Id NUMBER(5) PRIMARY KEY,
  Student_Id NUMBER(5) NOT NULL,
  Payment_Mode VARCHAR2(20)
    CHECK (Payment_Mode IN ('Cash', 'UPI', 'Card', 'Bank Transfer')),
  Payment_Status VARCHAR2(20)
    CHECK (Payment_Status IN ('Paid', 'Pending', 'Failed')),
  Amount NUMBER(10,2) CHECK (Amount > 0),
  CONSTRAINT fk_payment_student FOREIGN KEY (Student_Id)
    REFERENCES STUDENT(Student_Id)
);
```

8. Sample Data

The following sample data is fictional. Insert records in this order: HOSTEL, WARDEN, ROOM, STUDENT and PAYMENT. The complete insertion script is included below.

```
-- HOSTEL
INSERT INTO HOSTEL VALUES (1, 'Boys Hostel A', 'Campus Block A');
INSERT INTO HOSTEL VALUES (2, 'Girls Hostel B', 'Campus Block B');
INSERT INTO HOSTEL VALUES (3, 'Boys Hostel C', 'Campus Block C');

-- WARDEN: one hostel can have multiple wardens
INSERT INTO WARDEN VALUES (101, 'Rajesh Sharma', 45, '9876543210', 'Lucknow', 1);
INSERT INTO WARDEN VALUES (102, 'Amit Verma', 40, '9876543211', 'Kanpur', 1);
INSERT INTO WARDEN VALUES (103, 'Suresh Kumar', 48, '9876543212', 'Delhi', 2);
INSERT INTO WARDEN VALUES (104, 'Manoj Singh', 42, '9876543213', 'Lucknow', 2);
INSERT INTO WARDEN VALUES (105, 'Vikas Gupta', 39, '9876543214', 'Noida', 3);

-- ROOM
INSERT INTO ROOM VALUES (1, 101, 1, 2, 'AC');
INSERT INTO ROOM VALUES (1, 102, 1, 3, 'Non-AC');
INSERT INTO ROOM VALUES (1, 201, 2, 2, 'AC');
INSERT INTO ROOM VALUES (2, 101, 1, 2, 'Non-AC');
INSERT INTO ROOM VALUES (2, 102, 1, 3, 'AC');
INSERT INTO ROOM VALUES (3, 101, 1, 2, 'AC');

-- STUDENT
INSERT INTO STUDENT VALUES (201, 'Aarav Singh', 'CSE', 2, 8.50, DATE '2006-05-12', 1, 101);
INSERT INTO STUDENT VALUES (202, 'Priya Sharma', 'CSE', 2, 9.10, DATE '2006-08-15', 2, 101);
```

```
INSERT INTO STUDENT VALUES (203, 'Rahul Verma', 'IT', 3, 7.80, DATE '2005-04-20', 1, 101);
INSERT INTO STUDENT VALUES (204, 'Ananya Gupta', 'ECE', 1, 8.90, DATE '2007-01-10', 2, 102);
INSERT INTO STUDENT VALUES (205, 'Karan Mishra', 'CSE', 2, 8.20, DATE '2006-11-25', 3, 101);
INSERT INTO STUDENT VALUES (206, 'Neha Singh', 'IT', 3, 9.30, DATE '2005-09-18', 2, 102);

-- PAYMENT
INSERT INTO PAYMENT VALUES (301, 201, 'UPI', 'Paid', 45000);
INSERT INTO PAYMENT VALUES (302, 201, 'UPI', 'Paid', 5000);
INSERT INTO PAYMENT VALUES (303, 202, 'Cash', 'Paid', 45000);
INSERT INTO PAYMENT VALUES (304, 203, 'Card', 'Pending', 45000);
INSERT INTO PAYMENT VALUES (305, 204, 'UPI', 'Paid', 40000);
INSERT INTO PAYMENT VALUES (306, 205, 'Bank Transfer', 'Pending', 45000);
INSERT INTO PAYMENT VALUES (307, 206, 'UPI', 'Paid', 40000);

COMMIT;
```

9. SQL Queries

1. Student details with hostel and room

```
SELECT S.Student_Id, S.Student_Name, S.Course, H.Hostel_Name,
       R.Room_No, R.Floor_No
FROM STUDENT S
JOIN HOSTEL H ON S.Hostel_Id = H.Hostel_Id
JOIN ROOM R ON S.Hostel_Id = R.Hostel_Id
           AND S.Room_No = R.Room_No;
```

2. Hostel-wise wardens

```
SELECT H.Hostel_Name, W.Warden_Name, W.Phone_No
FROM HOSTEL H
JOIN WARDEN W ON H.Hostel_Id = W.Hostel_Id
ORDER BY H.Hostel_Id;
```

3. Total payment by each student

```
SELECT S.Student_Name, SUM(P.Amount) AS Total_Payment
FROM STUDENT S
JOIN PAYMENT P ON S.Student_Id = P.Student_Id
GROUP BY S.Student_Id, S.Student_Name;
```

4. Number of students in each hostel

```
SELECT H.Hostel_Name, COUNT(S.Student_Id) AS Total_Students
FROM HOSTEL H
LEFT JOIN STUDENT S ON H.Hostel_Id = S.Hostel_Id
GROUP BY H.Hostel_Id, H.Hostel_Name;
```

5. Students with pending payments

```
SELECT DISTINCT S.Student_Id, S.Student_Name
FROM STUDENT S
JOIN PAYMENT P ON S.Student_Id = P.Student_Id
WHERE P.Payment_Status = 'Pending';
```

6. Student(s) with highest CGPA

```
SELECT Student_Name, CGPA
FROM STUDENT
WHERE CGPA = (SELECT MAX(CGPA) FROM STUDENT);
```

7. Room occupancy and available seats

```
SELECT R.Hostel_Id, R.Room_No, R.Seater,
       COUNT(S.Student_Id) AS Occupied_Seats,
       R.Seater - COUNT(S.Student_Id) AS Available_Seats
FROM ROOM R
LEFT JOIN STUDENT S ON R.Hostel_Id = S.Hostel_Id
                   AND R.Room_No = S.Room_No
GROUP BY R.Hostel_Id, R.Room_No, R.Seater;
```

8. Total paid revenue

```
SELECT SUM(Amount) AS Total_Revenue
FROM PAYMENT
WHERE Payment_Status = 'Paid';
```

9. AC rooms

```
SELECT * FROM ROOM WHERE AC_or_NonAC = 'AC';
```

10. Students with CGPA above 8.5

```
SELECT Student_Name, CGPA FROM STUDENT WHERE CGPA > 8.5;
```

10. Testing and Validation

• Verify that all five tables are created and populated.

• Check primary and foreign keys using USER_CONSTRAINTS.

• Test duplicate primary keys and invalid foreign-key values with test records.

• Run joins and aggregate queries and compare outputs with sample records.

• Check room occupancy against Seater. The current occupancy query reports availability but does not itself prevent over-allocation.

Add screenshots here: successful table creation, inserted records, query outputs and constraint tests.

11. Limitations

• The current implementation focuses on the database layer and does not include a complete user interface.

• Room capacity is reported but is not automatically enforced by the current schema.

• There is no real payment gateway, authentication or audit logging.

12. Future Scope

• Develop a web or mobile interface.

• Add room-capacity validation using triggers or application logic.

• Maintain admission, checkout and room-transfer history.

• Generate fee receipts and payment summaries.

• Add role-based access for administrators, wardens and students.

13. Conclusion

The Hostel Management System demonstrates the design and implementation of a relational database using Oracle SQL. It organises hostel, warden, room, student and payment information into related tables and applies

keys and constraints to maintain referential integrity. Sample records and SQL queries demonstrate support for common hostel administration and reporting tasks.

15. References

• Oracle SQL documentation: https://docs.oracle.com/en/database/

• Oracle FreeSQL: https://freesql.com/

• DBMS course notes and classroom material.